Fathima Shafreena .F

192511381

EXPERIMENT 1: Linear Search (Iterative)

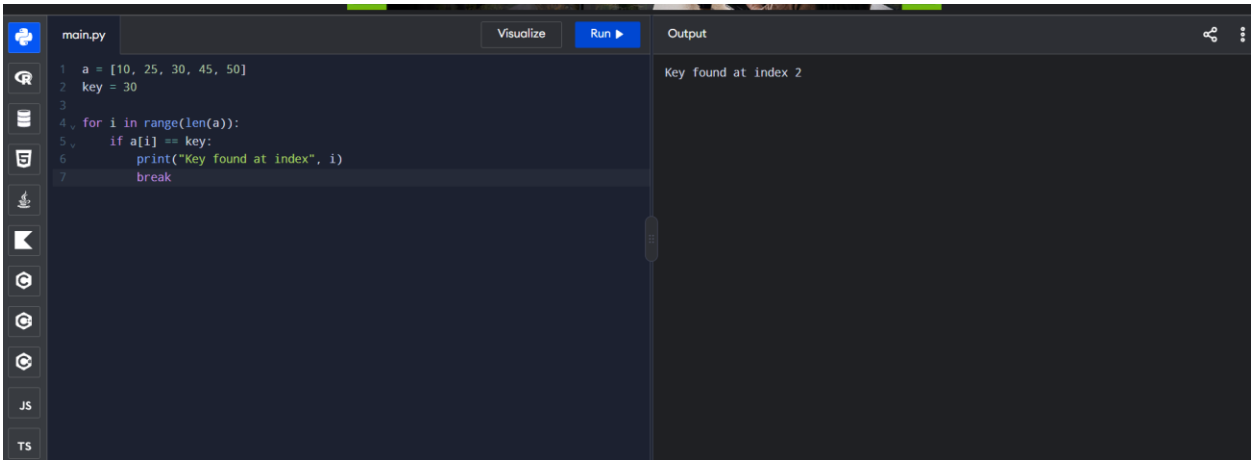
AIM:

To analyze the time complexity of linear search.

PSEUDO CODE:

1. Start.
2. Read the array and the key to be searched.
3. Compare the key with each element from the beginning of the array.
4. If the key matches an element, display its index and stop.
5. If the end of the array is reached, report that the key is not found.
6. Stop.

OUTPUT:



TIME COMPLEXITY:

Ω: O(1), O: O(n), Θ: Θ(n)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 2: Binary Search (Recursive)

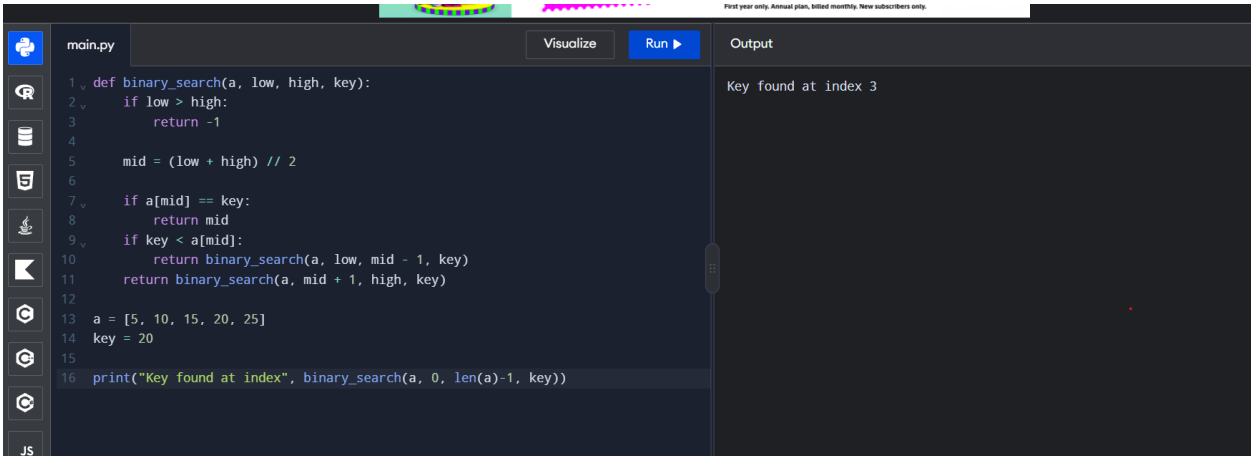
AIM:

To compare the complexity of recursive binary search.

PSEUDO CODE:

1. Start.
2. Read the sorted array and the key.
3. Find the middle element.
4. If the middle element is the key, return its index.
5. If the key is smaller, recursively search the left half.
6. If the key is larger, recursively search the right half.
7. Stop when the key is found or the search range becomes empty.

OUTPUT:



TIME COMPLEXITY:

Ω: O(1), O: O(logn), Θ: Θ(logn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 3: Factorial (Iterative vs Recursive)

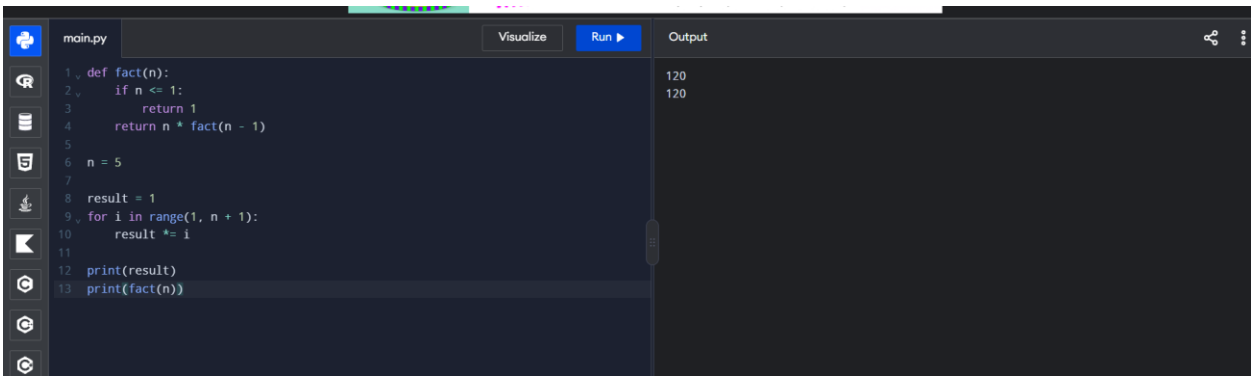
AIM:

To compare iterative and recursive factorial computation.

PSEUDO CODE:

1. Start.
2. Read the value of n.
3. For the iterative method, initialize the result to 1.
4. Multiply the result by each number from 1 to n.
5. For the recursive method, return 1 when n is 0 or 1.
6. Otherwise, return n multiplied by factorial of (n - 1).
7. Display the factorial result.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

Both O(n)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 4: Fibonacci Series

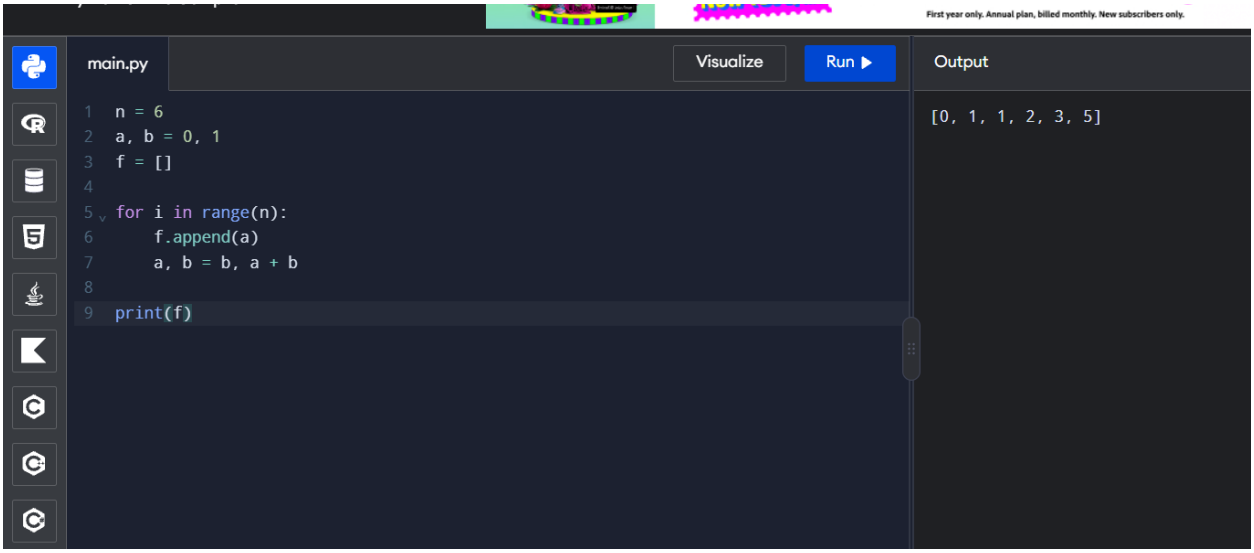
AIM:

To analyze recursive and iterative Fibonacci generation.

PSEUDO CODE:

1. Start.
2. Read n.
3. Initialize the first two Fibonacci values as 0 and 1.
4. Generate the next values by adding the previous two values.
5. Repeat until the first n Fibonacci numbers are generated.
6. Display the Fibonacci series.
7. Stop.

OUTPUT:



TIME COMPLEXITY:

Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 5: Matrix Multiplication

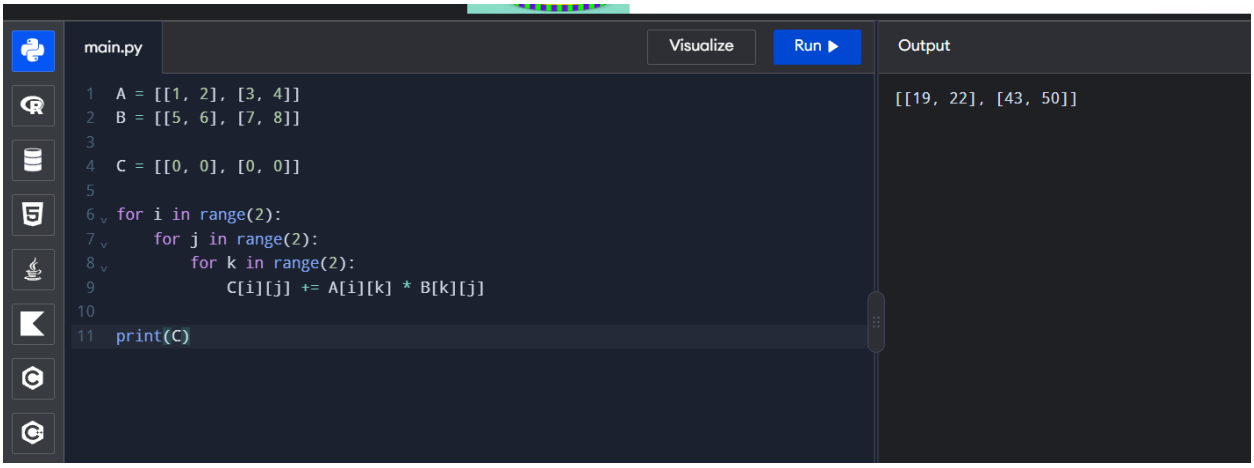
AIM:

To analyze the complexity of matrix multiplication using nested loops.

PSEUDO CODE:

1. Start.
2. Read matrices A and B.
3. Create a result matrix initialized with zeros.
4. For each row of A, select each column of B.
5. Multiply corresponding elements and add them to obtain each result element.
6. Repeat for all rows and columns.
7. Display the resulting matrix.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

O(n3)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 6: Merge Sort

AIM:

To analyze divide-and-conquer sorting using merge sort.

PSEUDO CODE:

1. Start.
2. Read the array.
3. If the array has one or zero elements, return it.
4. Divide the array into two halves.
5. Recursively sort both halves.
6. Merge the two sorted halves.
7. Display the sorted array.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

Always O(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 7: Quick Sort

AIM:

To analyze recursive quick sort.

PSEUDO CODE:

1. Start.
2. Read the array.
3. Select a pivot element.
4. Partition the array into elements smaller and larger than the pivot.
5. Recursively apply quick sort to the two parts.
6. Combine the sorted parts with the pivot.
7. Display the sorted array.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

Ω: O(nlogn), O: O(n2), Θ: Θ(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 8: Tower of Hanoi

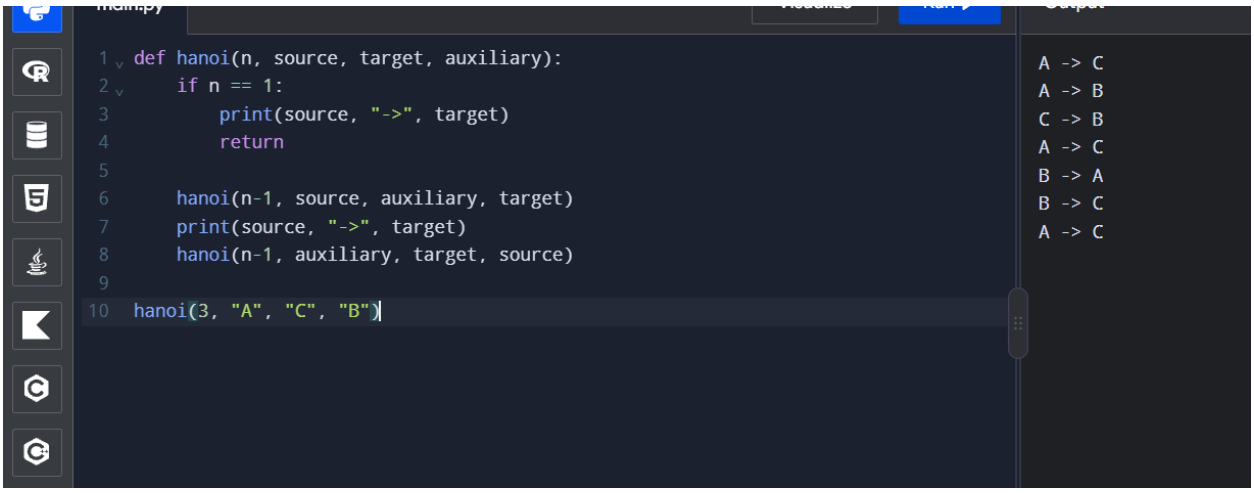
AIM:

To analyze exponential recursion using the Tower of Hanoi problem.

PSEUDO CODE:

1. Start.
2. Read the number of disks n.
3. If n is 1, move the disk from source to destination.
4. Recursively move n-1 disks from source to auxiliary.
5. Move the largest disk from source to destination.
6. Recursively move n-1 disks from auxiliary to destination.
7. Display the sequence of moves.
8. Stop.

OUTPUT:



```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(source, "->", target)
        return

    hanoi(n-1, source, auxiliary, target)
    print(source, "->", target)
    hanoi(n-1, auxiliary, target, source)

hanoi(3, "A", "C", "B")
```

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
```

TIME COMPLEXITY:

O(2n)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 9: GCD (Euclidean Algorithm)

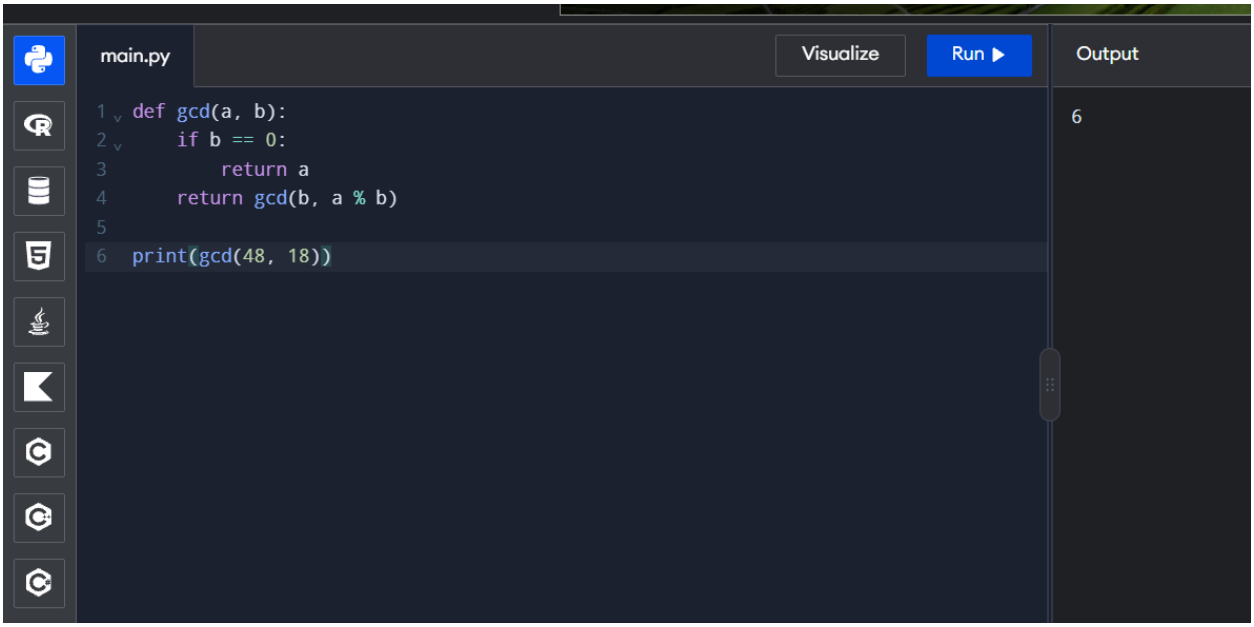
AIM:

To analyze recursive GCD using the Euclidean algorithm.

PSEUDO CODE:

1. Start.
2. Read a and b.
3. If b is 0, return a as the GCD.
4. Otherwise, find the remainder of a divided by b.
5. Recursively find GCD of b and the remainder.
6. Display the GCD.
7. Stop.

OUTPUT:



TIME COMPLEXITY:

O(logn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 10: Insertion Sort

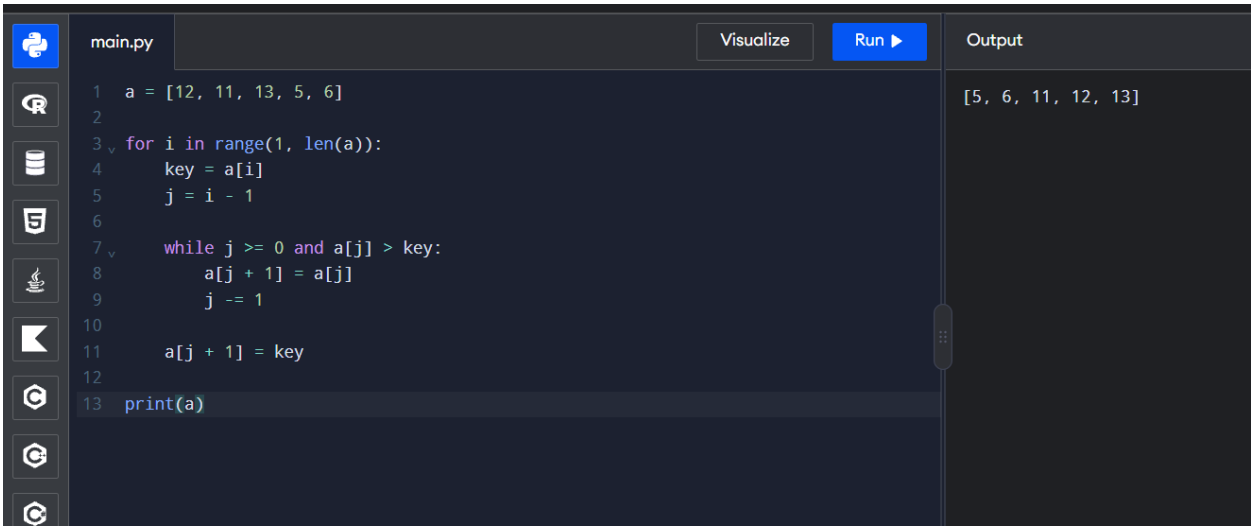
AIM:

To analyze iterative sorting using insertion sort.

PSEUDO CODE:

1. Start.
2. Read the array.
3. Select the next element as the key.
4. Compare the key with elements before it.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat until all elements are processed.
8. Display the sorted array.
9. Stop.

OUTPUT:



TIME COMPLEXITY:

Ω: O(n), O: O(n2), Θ: Θ(n2)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 11: Heap Sort

AIM:

To analyze heap-based sorting.

PSEUDO CODE:

1. Start.
2. Read the array.
3. Build a max heap from the array.
4. Swap the root element with the last element.
5. Reduce the heap size and restore the heap property.
6. Repeat until the array is sorted.
7. Display the sorted array.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

Always O(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 12: Counting Inversions

AIM:

To analyze a modified merge sort for counting inversions in an array.

PSEUDO CODE:

1. Start.
2. Read the array.
3. Divide the array into two halves.
4. Recursively count inversions in both halves.
5. Merge the two halves in sorted order.
6. During merging, count elements that form inversions.
7. Add all inversion counts.
8. Display the total number of inversions.
9. Stop.

OUTPUT:



TIME COMPLEXITY:

O(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 13: Maximum Subarray Sum

AIM:

To compare Kadane's algorithm with the divide-and-conquer method for finding the maximum subarray sum.

PSEUDO CODE:

1. Start.
2. Read the array.
3. For Kadane's method, initialize the current sum and maximum sum.
4. Traverse the array and update the current and maximum sums.
5. For divide-and-conquer, divide the array into two halves.
6. Find the maximum sum in the left half, right half, and crossing part.
7. Select the largest of the three sums.
8. Display the maximum subarray sum.
9. Stop.

OUTPUT:



TIME COMPLEXITY:

Kadane's O(n), Divide & Conquer O(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 14: Exponentiation

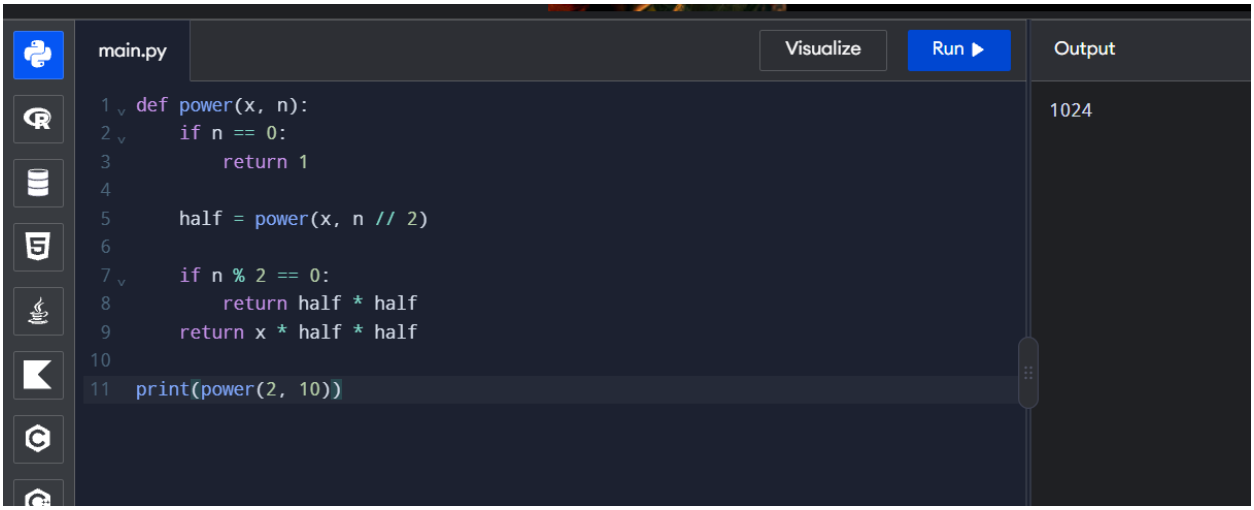
AIM:

To compare iterative and recursive fast power methods.

PSEUDO CODE:

1. Start.
2. Read x and n.
3. For the iterative method, multiply x by itself n times.
4. For recursive fast power, return 1 when n is 0.
5. Recursively calculate the power for half of n.
6. Square the result and multiply by x when n is odd.
7. Display the result.
8. Stop.

OUTPUT:



TIME COMPLEXITY:

Iterative O(n), Recursive fast power O(logn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.

EXPERIMENT 15: Closest Pair of Points

AIM:

To analyze the brute force and divide-and-conquer approaches for finding the closest pair of points in 2D.

PSEUDO CODE:

1. Start.
2. Read the set of points.
3. For the brute-force method, calculate the distance between every pair of points.
4. Keep the pair with the smallest distance.
5. For the optimized method, divide the points into two parts.
6. Find the closest pair in each part and compare the relevant points across the division.
7. Select the pair with the minimum distance.
8. Display the closest pair and its distance.
9. Stop.

OUTPUT:



TIME COMPLEXITY:

Brute force O(n2), Optimized O(nlogn)

RESULT:

Thus, the given algorithm was implemented and its time complexity was analyzed successfully.